Deep Learning

01-06-26

Activation Functions:

- Activation function is used to generate output of a neuron after weighted aggregation of inputs.



$$a^l = f(z^l)$$
$$z^l = w^l a^{l-1}$$

- Activation functions are very very important to learn various types of relationships between inputs and output

↓

Linear + non-linear relationships

- Activation functions are essential for non-linear relationships in ANN.

- with weighted aggregation only - ANN can only learn linear relationships.

Types of Activation Functions:

1) Linear Activation

$$f(x) = x$$

$f(2) = 2$ $\qquad a = f(z)$

$f(-4) = -4$ $\qquad a = z$

Usage:

- used mostly in last layer of ANN regression models
- rarely used in intermediate hidden layers

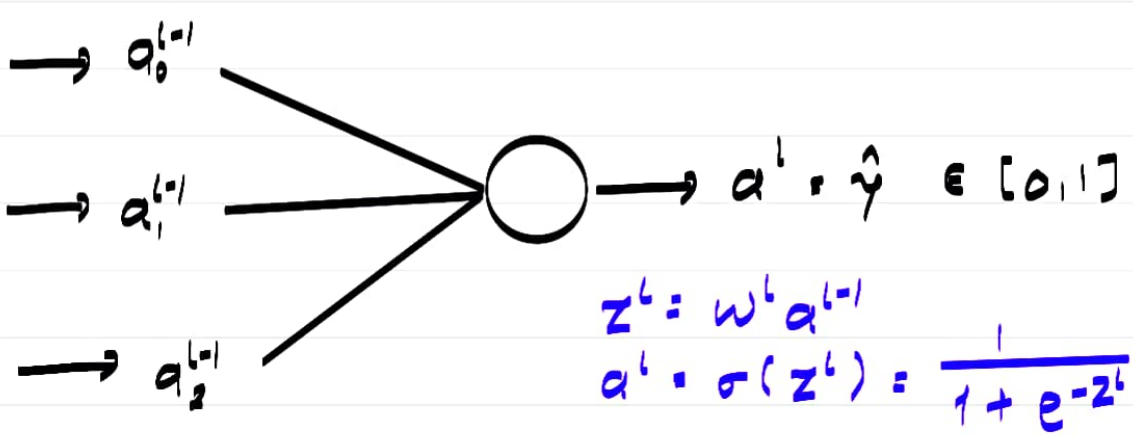
2) Sigmoid Activation

$$f(x) = \sigma(x) = \frac{1}{1 + e^{-x}}$$

output values have range $[0, 1]$

Usage:

- widely used in output layer of ANN binary classification model.



- sigmoid activation is very very rare in intermediate hidden layers but widely used in output layer

3) ReLU Activation

- Rectified Linear Unit

$$f(x) = relu(x) = max(0, x)$$

relu(5) = max(0,5) = 5
relu(1) = max(0,1) = 1
relu(-2) = max(0,-2) = 0
relu(-0.005) = 0

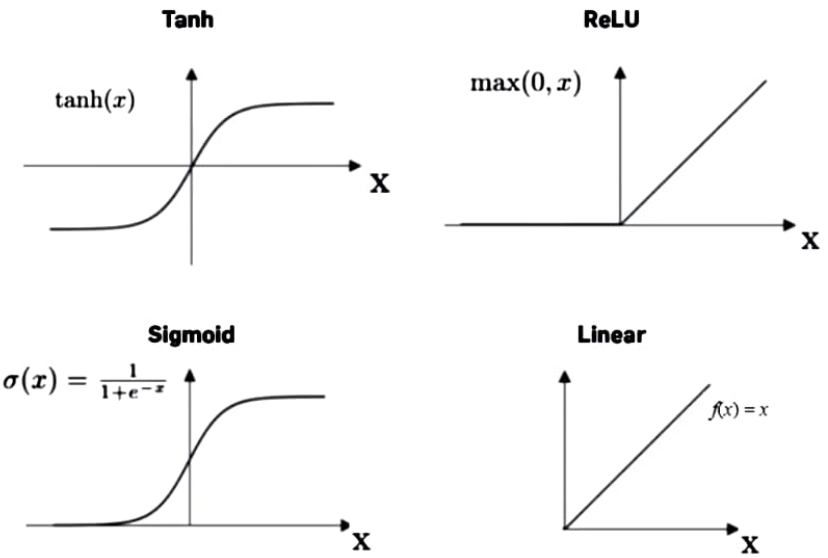
} gives output as 0 if input is negative

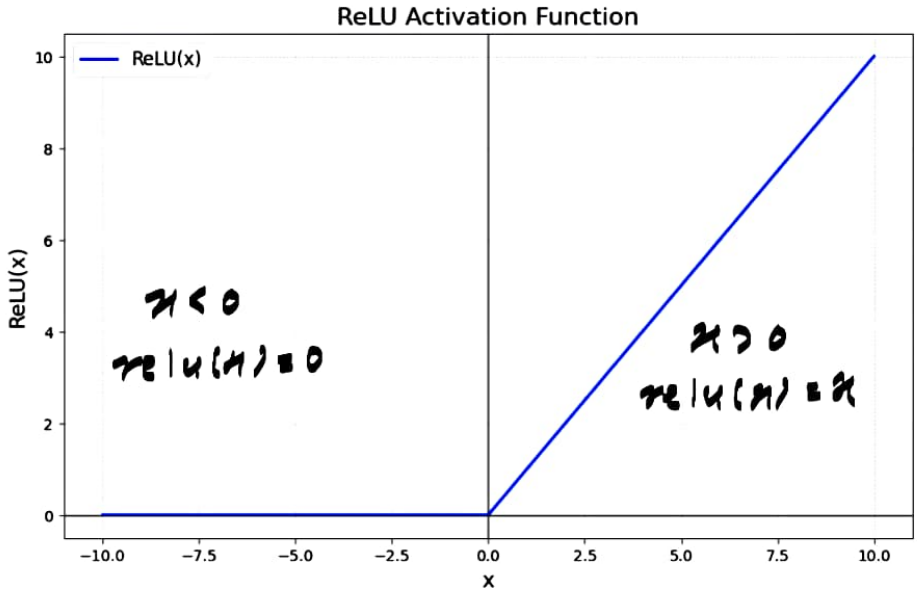
} gives linear output if input is positive

- output range is 0 to $+\infty$

Usage

- widely used in the intermediate hidden layers
- rarely used in output layer



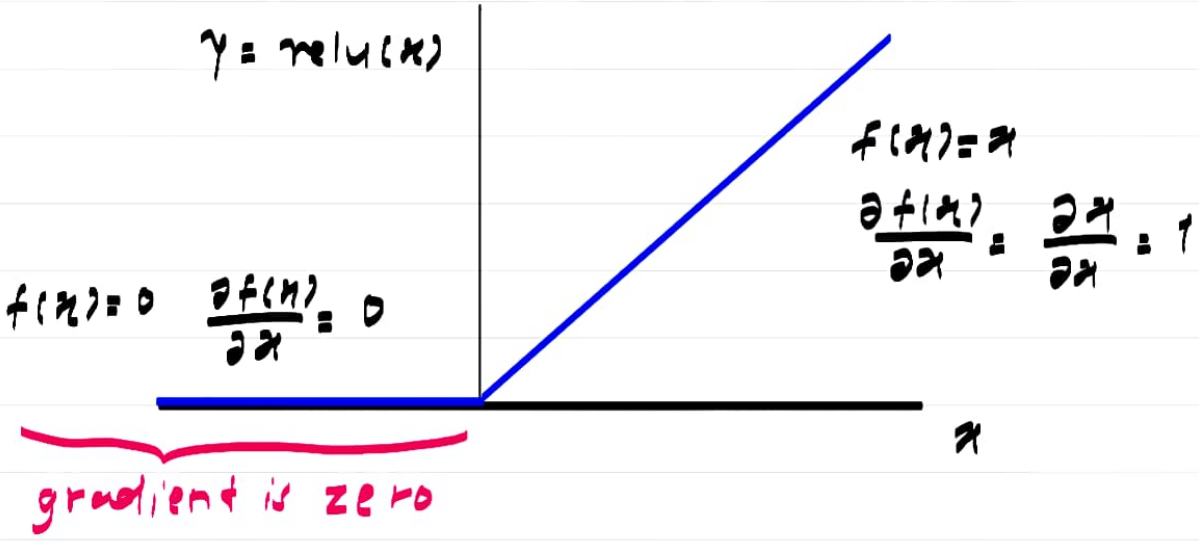


is relu a linear function? → No.

- relu is piecewise linear function, but overall its a non-linear function which can learn non-linear relationships

- Relu is very efficient, fast and easy to compute compared to other non-linear activations

- Relu activation faces issue of zero gradients. This issue is called dying relu issue.



If any neuron having relu activation gets negative input then the training of that neuron stops due to dying relu issue

neron → has weights
↓
trained / updated using gradient descent

gradient descent → uses gradients
↓
if gradients are zero
↓
neuron training has stopped ← no weight updation

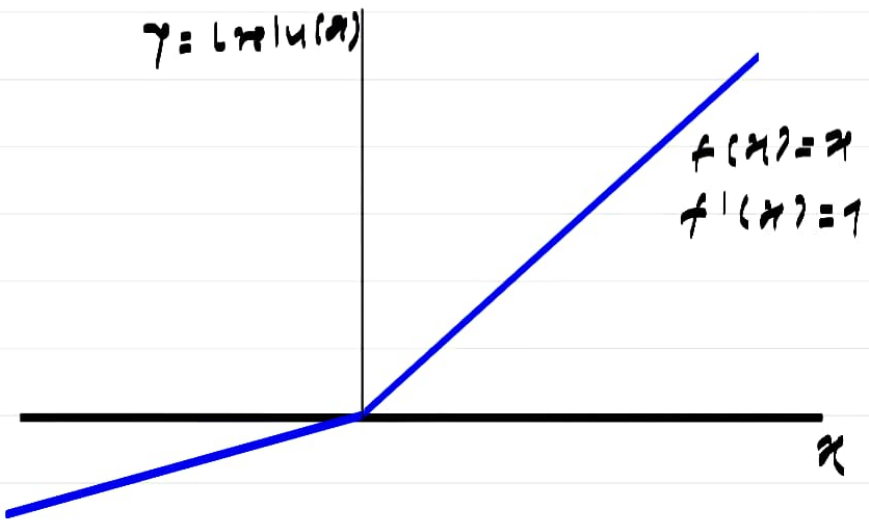
$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w_{old}}$$

To solve this problem, we modified relu and created its variants

Leaky Relu Activation

$f(x) = x$ if $x \geq 0$
$= \alpha x$ if $x < 0$

where $\alpha < 1$



$\frac{\partial f(x)}{\partial x} = f'(x) = \alpha$ $f(x) = \alpha x$

α - very small value lesser than 1

for $\alpha = 0.2$

Lrelu(5) = 5

Lrelu(1) = 1

Lrelu(-2) = αx = 0.2(-2) = -0.4

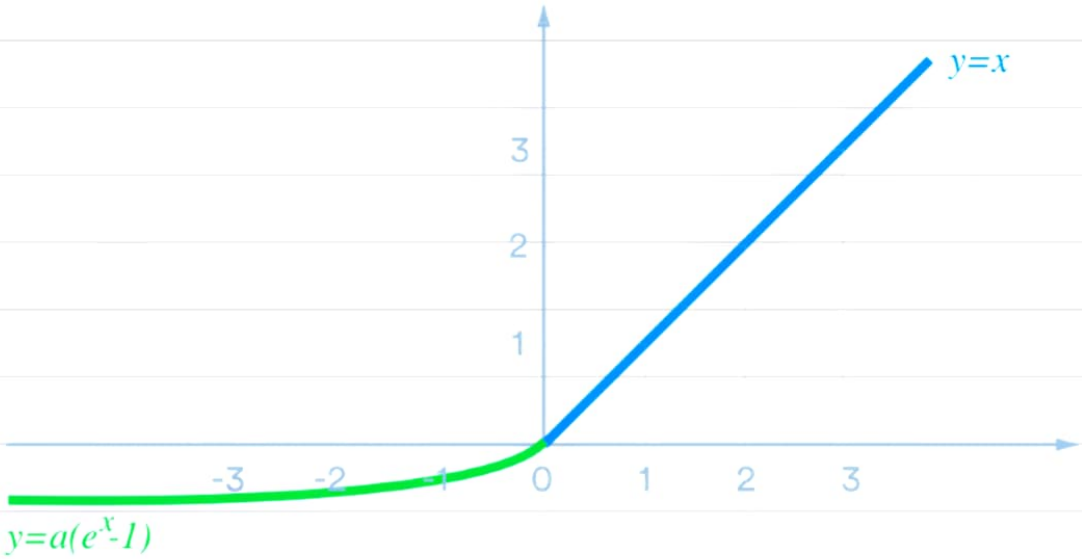
Lrelu(-10) = αx = 0.2(-10) = -2

as $\frac{\partial f(x)}{\partial x} \neq 0$ in negative inputs of x, dying relu issue gets solved.

Exponential Linear Unit (ELU)

$$ELU(x) = x \text{ if } x \geq 0$$
$$= \alpha(e^x - 1) \text{ if } x < 0$$

α is a small +ve constant

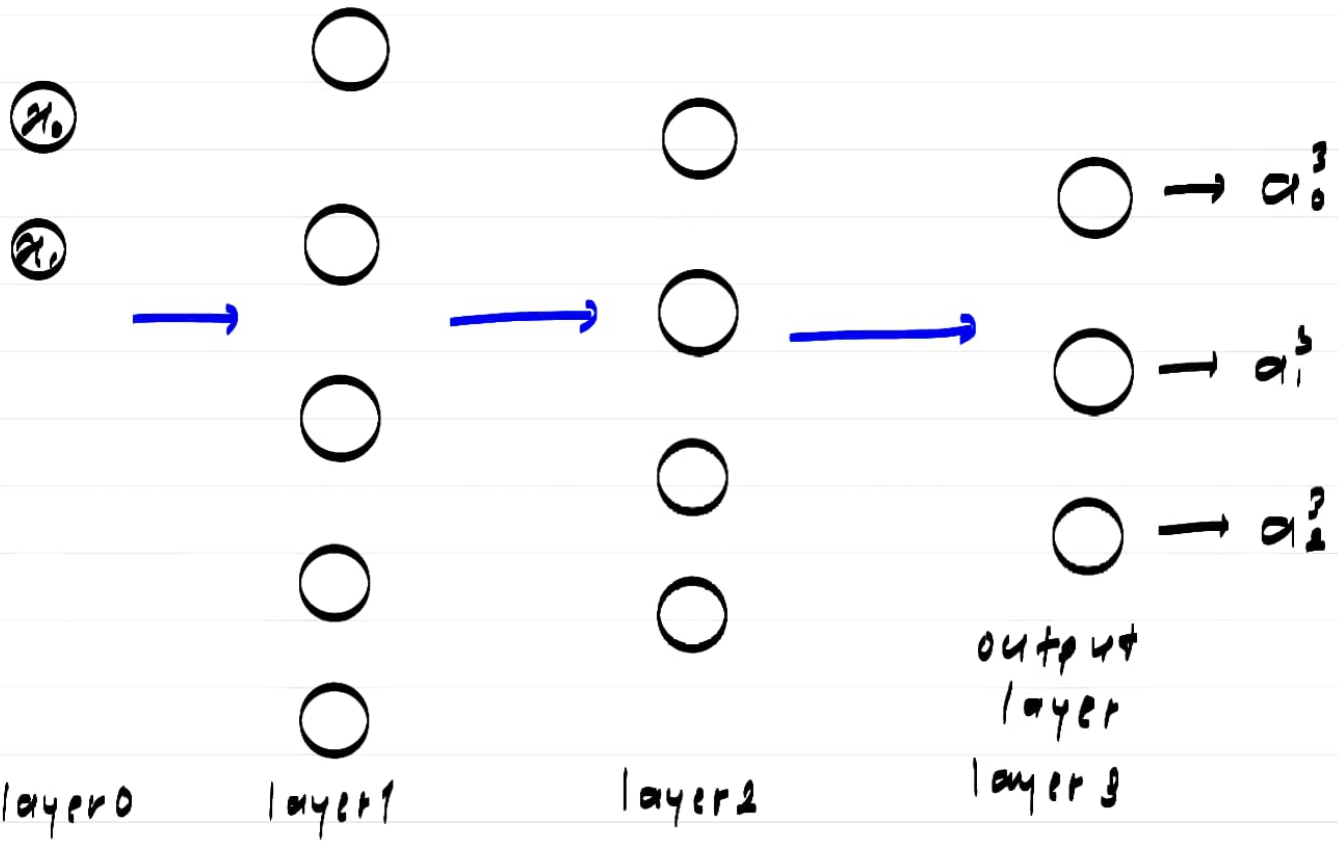


- ELU adds more non-linearity to the activation
- this means ELU can learn non-linear relationships even better than relu or Leaky relu.

Softmax Activation

used in multiclass classification ANN

eg. ANN multiclass clf with number of classes = 3

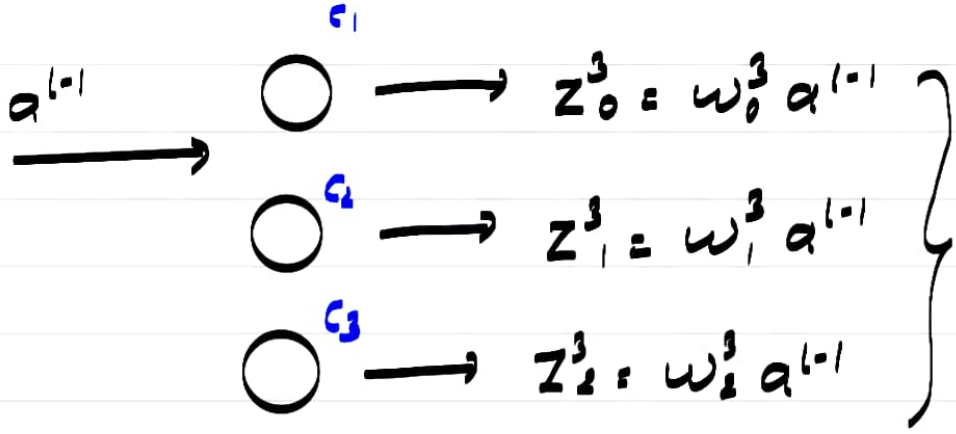


softmax activation takes multiple inputs and gives multiple outputs

In multiclass clf ANN, the number of nodes in output layer, must be equal to number of unique classes in target

3 classes = 3 nodes in output layer

each node will predict probability of its associated class.



$$\text{softmax}(z_0^3, z_1^3, z_2^3) : \frac{e^{z_0^3}}{\sum_{i=1}^{3} e^{z_i^3}}, \frac{e^{z_1^3}}{\sum_{i=1}^{3} e^{z_i^3}}, \frac{e^{z_2^3}}{\sum_{i=1}^{3} e^{z_i^3}}$$

$$\sum_{i=1}^{3} e^{z_i^3} : e^{z_0^3} + e^{z_1^3} + e^{z_2^3} : \sum e^z$$

$\frac{e^{z_0^3}}{\sum e^z}$ → prob. of c_1

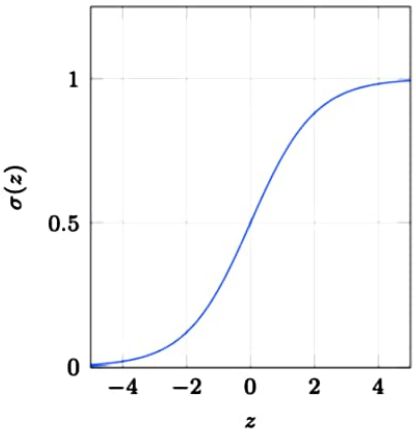
$\frac{e^{z_1^3}}{\sum e^z}$ → prob of c_2

$\frac{e^{z_2^3}}{\sum e^z}$ → prob of c_3

} final model prediction is the class with highest predicted probability.

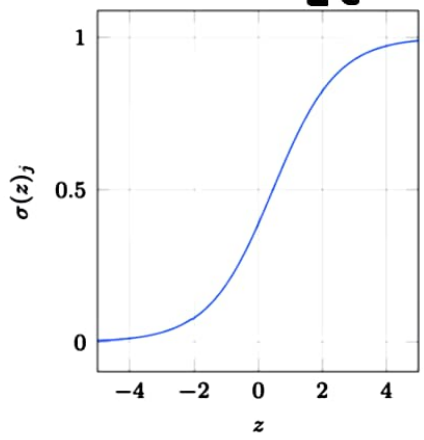
- softmax is commonly used in the output layer of multiclass classification ANN.
- rarely used in intermediate hidden layers.

$f(x) = \frac{1}{1+e^{-x}}$



(a) Sigmoid activation function.

$f(x) = \frac{e^{x}}{\sum e^{x}}$



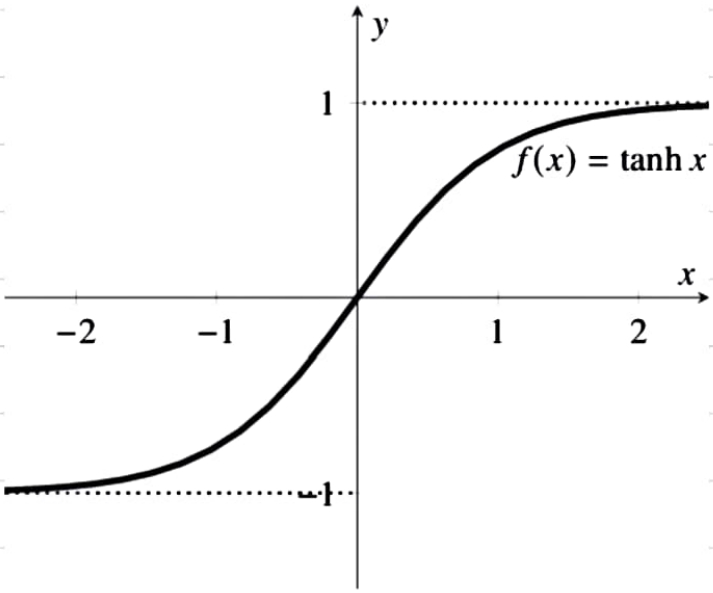
(b) Softmax activation function.

Figure 1: Sigmoid and Softmax activation functions

Tanh Activation :

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

output range = -1 to +1



- commonly used in intermediate hidden layers and rarely used in output layer
- outputs of tanh are zero centered which helps gradient descent in model training